ПРОГРАММИРОВАНИЕ НА PYTHON

Практическое задание

Разработка CLI-утилит с использованием ИИ-инструментов

Александр Тимофеев

Обзор практического задания

Цель задания

Освоить процесс разработки CLI-утилит с использованием современных ИИ-инструментов, научиться сравнивать и анализировать разные подходы к генерации кода.

Срок выполнения: 6-8 недель

Этап 1	Этап 2	Этап 3	Этап 4	Этап 5
Выбор темы	Подготовка ТЗ	Разработка	Отчет	GitHub
1-2	*2-3*	*3-6*	*6-7*	*7-8*

Этап 1: Выбор темы и согласование

Задача

Выбрать тему CLI-утилиты и согласовать с преподавателем.

Требования к теме

- Практическая полезность
- Средняя или высокая сложность
- Возможность реализации за 4-6 недель
- Использование минимум 7 команд/подкоманд
- **Обязательное использование REST API (внешнего или собственного)**

✓ Результат: Утвержденная тема + краткое описание (1 абзац)

Обязательное требование: REST API

Ваша CLI-утилита ОБЯЗАТЕЛЬНО должна использовать REST API

Вариант 1: Внешние API

- GitHub API, OpenWeather API
- Stripe API, Notion API
- Spotify API, Twitter/X API
- Google Calendar API
- Any public REST API

Вариант 2: Собственный API

- Создать FastAPI/Flask сервер
- Поднять локально или на хостинге
- Описать endpoints в README
- CLI взаимодействует с вашим API

⚠️ **Утилита без REST API НЕ будет принята к защите!**

Примеры тем для проекта (с REST API)

Password Manager CLI

С собственным FastAPI бэкендом

GitHub Issues CLI

Использует GitHub REST API

Weather Dashboard CLI

OpenWeather + визуализация

Task Manager CLI

Todoist/Notion API интеграция

Network Monitor

Свой API для сбора метрик

Crypto Portfolio CLI

CoinGecko/Binance API

ML Model Deployer

Свой API + управление моделями

Social Media CLI

Twitter/Reddit API клиент

Этап 2: Подготовка ТЗ совместно с ИИ

Задача: С помощью любой нейросети составить детальное ТЗ

Функциональные требования

- Список команд и подкоманд
- Аргументы и опции
- Примеры использования
- Ожидаемое поведение

Технические требования

- Используемые библиотеки
- Структура проекта
- Обработка ошибок
- REST API endpoints

Дополнительно

- Документация API
- Тестирование
- Конфигурация
- Логирование

Этап 3: Параллельная разработка

Реализовать проект в двух разных нейросетях одновременно

Выбор нейросетей (2 из списка)

- ChatGPT
- Claude (Anthropic)
- Cursor AI
- Gemini (Google)
- GitHub Copilot
- Grok
- DeepSeek
- Локальные модели

Ведение параллельных сессий

- Сохранять все промпты и ответы
- Фиксировать время и количество итераций
- Отмечать проблемы и эмоции
- Документировать решения

Результат: Два рабочих проекта

Этап 4: Подготовка отчета (часть 1)

Введение

- Описание проекта
- Выбранные ИИ-инструменты
- Ожидания от сравнения

Методология

- Как велось сравнение
- Критерии оценки
- Процесс документирования

Разработка в Нейросеть 1

- Все промпты с таймстампами
- Проблемы и решения
- Особенности взаимодействия

Разработка в Нейросеть 2

- Аналогично предыдущему разделу
- Сохранение всех деталей

Этап 4: Подготовка отчета (часть 2)

Сравнительный анализ и выводы

Критерии сравнения

- Понимание контекста
- Качество кода
- Поддержка диалога
- Скорость ответа
- Обработка ошибок
- Креативность решений
- Документирование
- Производительность

Выводы и рекомендации

- Какой инструмент лучше для каких задач
- Советы по промпт-инжинирингу
- Рекомендации для будущих проектов
- Сравнение эффективности

Формат: Детальный отчет Markdown

Этап 5: Публикация на GitHub

Разместить оба проекта на GitHub с качественной документацией

Требования к README

- Описание проекта и API
- Установка и использование
- Сравнение подходов
- Скриншоты работы
- Инструкция по тестированию

Структура репозитория

```
├── version1/ (нейросеть 1)
├── version2/ (нейросеть 2)
├── docs/ (документация)
├── tests/ (тесты)
├── README.md
└── comparison.md (отчет)
```

Результат: Публичный GitHub репозиторий с обеими версиями

Критерии оценивания практической работы

Техническая часть (40%)

- Работоспособность обеих версий
- Соответствие ТЗ
- Качество кода (PEP 8)
- Обработка ошибок
- Использование REST API

Процесс разработки (30%)

- Качество промптов
- Эффективность с ИИ
- Решение проблем
- Документирование процесса

Аналитическая часть (20%)

- Глубина сравнения
- Практические выводы
- Качество отчета
- Объективность

Документация (10%)

- Качество README
- Структура репозитория
- Четкость инструкций
- Оформление

Частые ошибки и как их избежать

❌ **Забыли про REST API**

✓ CLI обязательно должна использовать API!

❌ **Потеря промптов**

✓ Сразу сохраняйте все диалоги в отдельный файл

❌ **Поверхностное сравнение**

✓ Анализируйте конкретные метрики и примеры

❌ **Неполное ТЗ**

✓ Детально опишите все команды и API endpoints

❌ **Копирование кода**

✓ Работайте независимо с каждой нейросетью

❌ **Отсутствие тестов**

✓ Напишите минимум 5-10 unit-тестов

Чек-лист перед сдачей

Убедитесь, что все пункты выполнены:

- ☐ CLI использует REST API (внешний или собственный)
- ☐ Обе версии утилиты работают без ошибок
- ☐ Все команды реализованы согласно ТЗ
- ☐ Код соответствует PEP 8 стандартам
- ☐ Написаны unit-тесты (покрытие > 60%)
- ☐ README содержит описание API, установку, примеры
- ☐ Сохранены все промпты и диалоги
- ☐ Отчет включает детальное сравнение
- ☐ Репозиторий имеет правильную структуру
- ☐ Добавлены badges, лицензия, .gitignore

Успехов в разработке!

Помните: качество промптов = качество кода

Вопросы? Обращайтесь:
Александр Тимофеев